To-do List

Rapport de projet

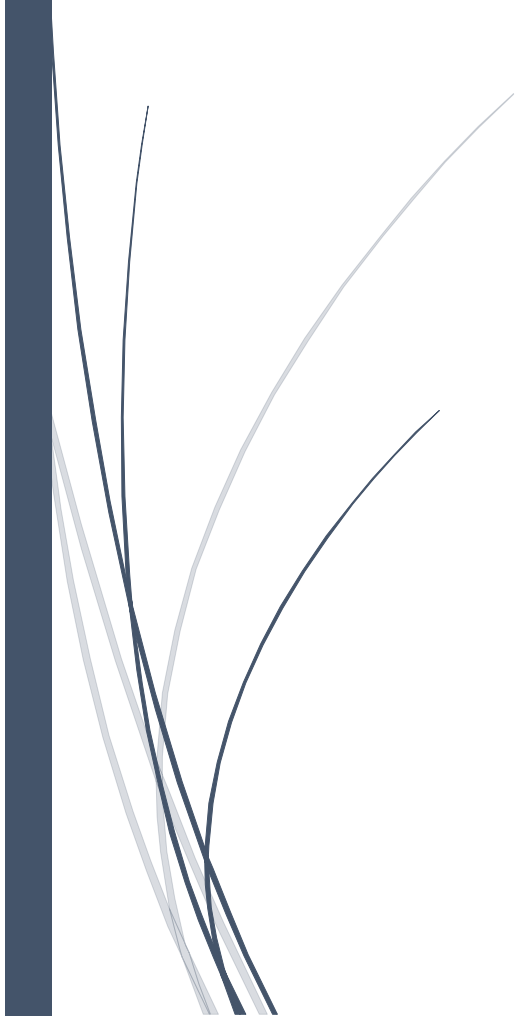
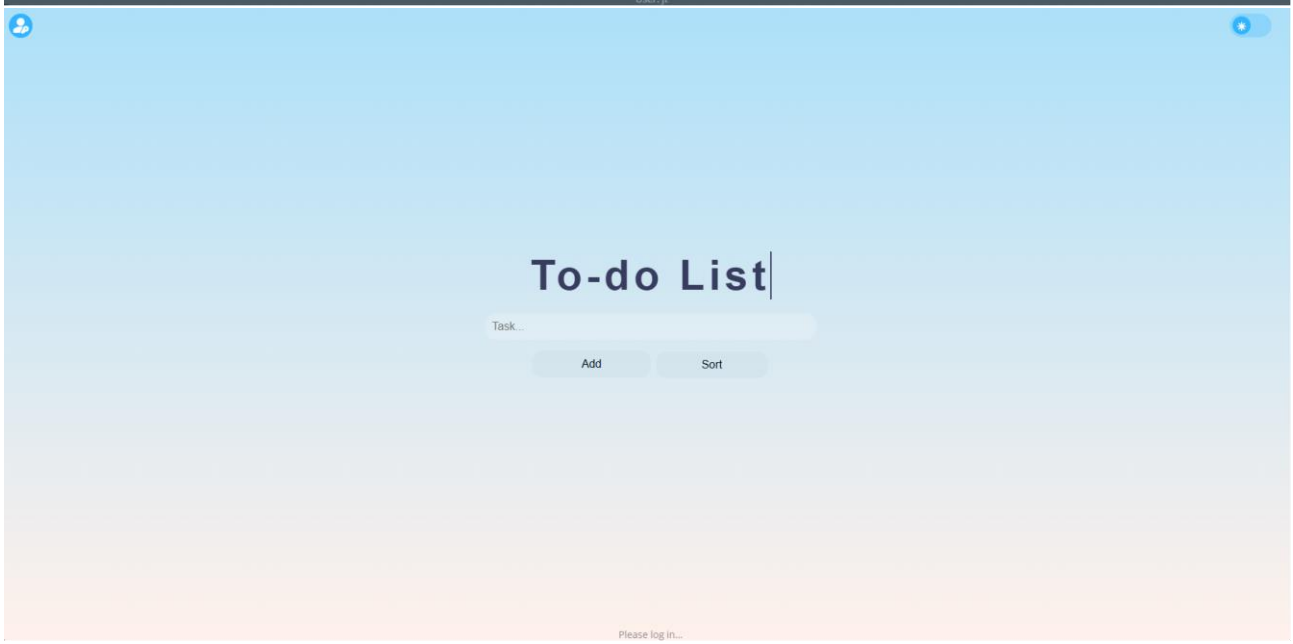


Table des matières

Résumé du projet 2
Langage utilisé 3
Interface de l'utilisateur 3
Arborescence du Projet 3
Présentation et explication du code 3
HTML 4
SQL 5
PHP 6
JavaScript 7
Dark/Light mode 7
Fonction enter 7
Ajouter une tâche 8
Récupérer les tâches 9
Tri des tâches & affichage 10
Modifier une tâche 10
Supprimer une tâche 11
Création d'un compte 11
Login 12
Log out 13
Conclusion 14


To-do List
Task...
Add
Sort
Task 1
User: jt


To-do List
Task...
Add
Sort
Please log in...

Résumé du projet

Développement d'un site web simple, intuitif et fonctionnel de gestion de tâches (To-Do List), destiné à un usage personnel.

Ce projet permet de :

- Ajouter une tâche ;
- Modifier une tâche ;
- Supprimer une tâche ;
- Marquer une tâche comme terminée ;
- Changer la couleur du thème du site (thème clair/sombre) ;
- Définir l'importance d'une tâche ;
- Définir la date de fin et de début d'une tâche ;
- Organiser les tâche (par défaut, la dernière additionner, par date & par importance) ;
- Système de login/log out et permettre la création de nouveau comptes ;

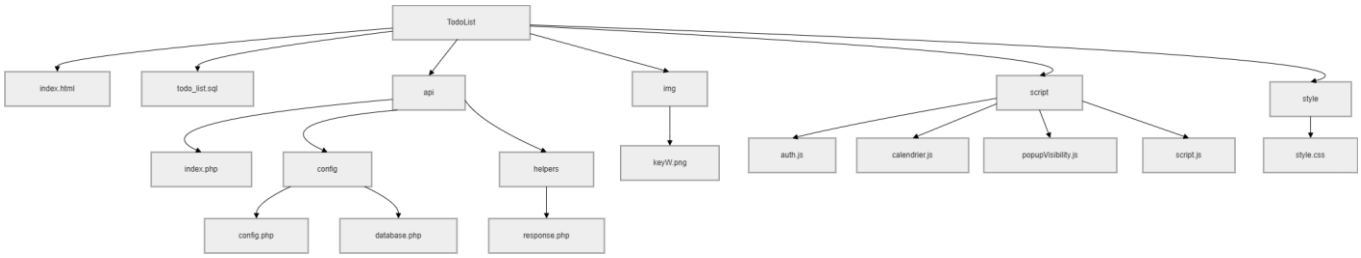
Langage utilisé

Le langage utilisé pour programmer le site web a été HTML, CSS, JavaScript, PHP et SQL.

Interface de l'utilisateur

- Champ de saisie et un bouton (« add ») pour ajouter une tâche ;
- Liste des tâches affichées sous le champ de saisie ;
- Changer le statut des tâches (terminer/en cours) ;
- Boutons pour modifier ou supprimer chaque tâche ;
- Thèmes sélectionnables (clair, sombre) ;
- Un bouton pour pouvoir login/crée un compte et log out ;
- Un bouton de filtrage (ordre par défaut, dernière tâche ajoutée, niveau d'importance et date).

Arborescence du Projet



Présentation et explication du code

HTML

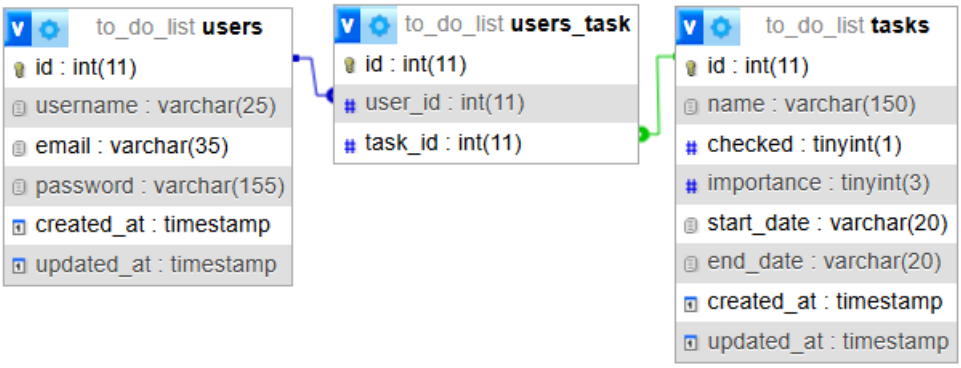
Interface principale :

- Un switch (bouton bascule) qui permet de changer le thème (clair/sombre) (qui appelle la fonction toggleDarkMode()) ;
- Un titre principal (“To-do List”) ;
- Un champ de saisie (input) principal pour écrire le une nouvelle tâche ;
- Un bouton d’ajout (“Add”) de tâche (appelle la fonction addTask()) ;
- Un menu déroulant (“Sort”) pour trier les tâches (appelle la fonction dropdownVisualisation() au clic) ;
- Une liste <ul> (id="AddTache") où les tâches seront injectées dynamiquement par le JavaScript ;
- Un bouton login (appelle la fonction hideShow()) ;
- Un espace en bas de page pour afficher le nom de l'utilisateur connecté (userLabel) et un bouton de déconnexion (logoutS) ;
- Un bloc caché (alertPopup) qui affiche les notifications/alertes personnalisées de l'application.

Popups et formulaires (cachés par défaut) :

- Popup de Connexion : Un formulaire contenant des champs de saisie (nom d'utilisateur et mot de passe), un bouton de validation, et un lien appelant createAccountPopUp() pour basculer vers la création de compte ;
- Popup de Création de compte : Un formulaire contenant des champs de saisie (email, mot de passe, nom d'utilisateur), un bouton de validation, et un lien appelant creatToLoginPopup() pour revenir à la connexion ;
- Popup d'Édition de tâche : formulaire de modification contenant :
    - Un champ de texte pour renommer la tâche ;
    - Des boutons de type "radio" pour redéfinir le niveau d'importance (High, Mid, Low) ;
    - Deux champs de saisie pour les dates (Début et Fin) qui appellent la fonction srtCal() pour ouvrir le calendrier au clic ;
    - Un bouton d'édition pour sauvegarder les changements ;
    - Un bouton de fermeture (croix) appelant la fonction closeModifTask().
- Interface du Calendrier : Un bloc interactif contenant l'affichage du mois/année, des boutons de navigation (flèches), et une grille où les jours sont générés par le code JS ;
- Arrière-plans interactifs : Des blocs invisibles ou floutés en arrière-plan des popups, qui permettent de fermer les fenêtres lorsqu'on clique à l'extérieur (ils appellent des fonctions comme zIndexBetweeenLoginTodo(), closeModifTask(), etc.).

SQL



- Table users : Gère les comptes. Elle stocke l'identifiant unique (ID), le nom d'utilisateur, l'email et le mot de passe de manière sécurisée (haché).
- Table tasks : Contient toutes les informations propres à une tâche : son nom, son statut (cochée ou non via un booléen checked), son niveau d'importance, et ses dates de début et de fin.
- Table de liaison users_task : C'est une table intermédiaire qui relie un utilisateur (ID) à ses tâches (ID). Elle utilise des clés étrangères (FOREIGN KEY) avec la contrainte ON DELETE CASCADE, ce qui signifie que si un compte utilisateur est supprimé, toutes ses tâches disparaîtront automatiquement de la base de données.

PHP

- **Dossier config** (config.php & database.php) : Contient les identifiants de connexion à la base de données et établit la connexion sécurisée grâce à l'interface PDO (PHP Data Objects).
- **Dossier helpers** (response.php) : Contient une fonction utilitaire (jsonResponse) qui permet de formater automatiquement toutes les réponses du serveur au format JSON, lisible par le JavaScript.
- **Routeur principal** (index.php) : Analyse l'URL demandée et la méthode HTTP (GET, POST, PUT, DELETE) pour exécuter la bonne action en base de données.

Fonctionnalités principales de l'API:

- Gestion des utilisateurs (/api/users) : Permet l'inscription de nouveaux membres. Le script vérifie d'abord si le nom d'utilisateur est déjà pris pour éviter les doublons (retourne une erreur 409 Conflict si c'est le cas).
- Gestion des tâches (/api/users/:id/tasks) : Gère le CRUD des tâches. Permet de récupérer (GET) uniquement les tâches liées à l'utilisateur connecté, d'en créer de nouvelles (POST), de les modifier (PUT) ou de changer leur statut coché/décoché (PATCH).
- Authentification (/api/login) : Reçoit le pseudo et le mot de passe tapés par l'utilisateur, cherche le compte correspondant en base de données, et vérifie si le mot de passe est correct.

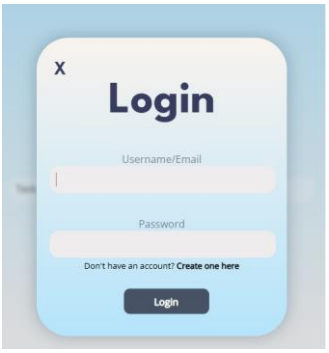
JavaScript

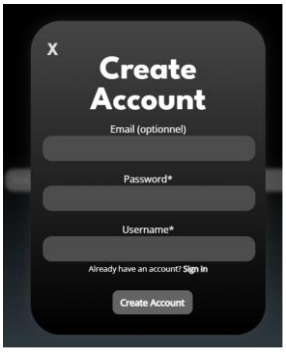
Dark/Light mode

```js
function toggleDarkMode() {
  const isDark = document.body.classList.toggle("dark-mode");
  localStorage.setItem("theme", isDark ? "dark" : "light");
}
```










```html
<div class="ButtonDark" id="btnToggleDark" onclick="event.stopPropagation()">
    <label class="switch">
      <input type="checkbox" id="darkToggle" onchange="toggleDarkMode()" />
      <span class="BTNDark"></span>
    </label>
  </div>
```

(html)

- **HTML :** Un bouton interrupteur (checkbox) appelle la fonction toggleDarkMode() à chaque clic.
- **JavaScript :** La fonction bascule l'affichage en mode sombre (ajout de la classe CSS) et sauvegarde le choix de l'utilisateur dans son navigateur (localStorage) pour ses prochaines visites.
- **Le design :** L'aperçu visuel du bouton bascule et change le thème.

Fonction enter

```js
function enterKey(inputId, btnId) {
  const inputElement = document.getElementById(inputId);
  if (!inputElement) return;
  inputElement.addEventListener("keypress", function (event) {
    if (event.key === "Enter") {
      event.preventDefault();
      document.getElementById(btnId).click();
    }
  });
}

enterKey("myInput", "addBtn");
enterKey("textTaskModif", "modifBtn");
```

- Permet à l'utilisateur de valider son texte directement au clavier, sans avoir à utiliser sa souris pour cliquer sur les boutons. C'est une fonction générique et est appelée deux fois dans le code (pour le champ "Ajouter une tâche" et une fois pour le champ "Modifier la tâche").
- **JavaScript :** La fonction enterKey écoute les frappes sur un champ de texte (keypress). Si elle détecte la touche "Enter", elle bloque le rechargement de la page (preventDefault) et déclenche un faux clic (click()) sur le bouton associé.

Ajouter une tâche



```
async function addTask() {
  const input = document.getElementById("myInput");
  const inputValue = input.value.trim();
  const userId = localStorage.getItem("currentUserId");

  if (inputValue === "") {
    showAlert("You must write something!");
    return;
  }
  if (!userId) {
    showAlert("You must be logged in to add a task...");
    return;
  }

  // Formatage de la date du jour: DD/MM/YYYY
  const today = new Date();
  const formatedToday = `${String(today.getDate()).padStart(2, "0")}/${String(today.getMonth() + 1).padStart(2, "0")}/${today.getFullYear()}`;

  const newTaskData = {
    name: inputValue,
    startDate: formatedToday,
    endDate: "",
    importance: 1,
  };

  const response = await apiCall(
    "/users/" + userId + "/tasks",
    "POST",
    newTaskData,
  );

  if (response) {
    input.value = "";
    await render(); // Rafraîchit l'affichage
  }
}
```

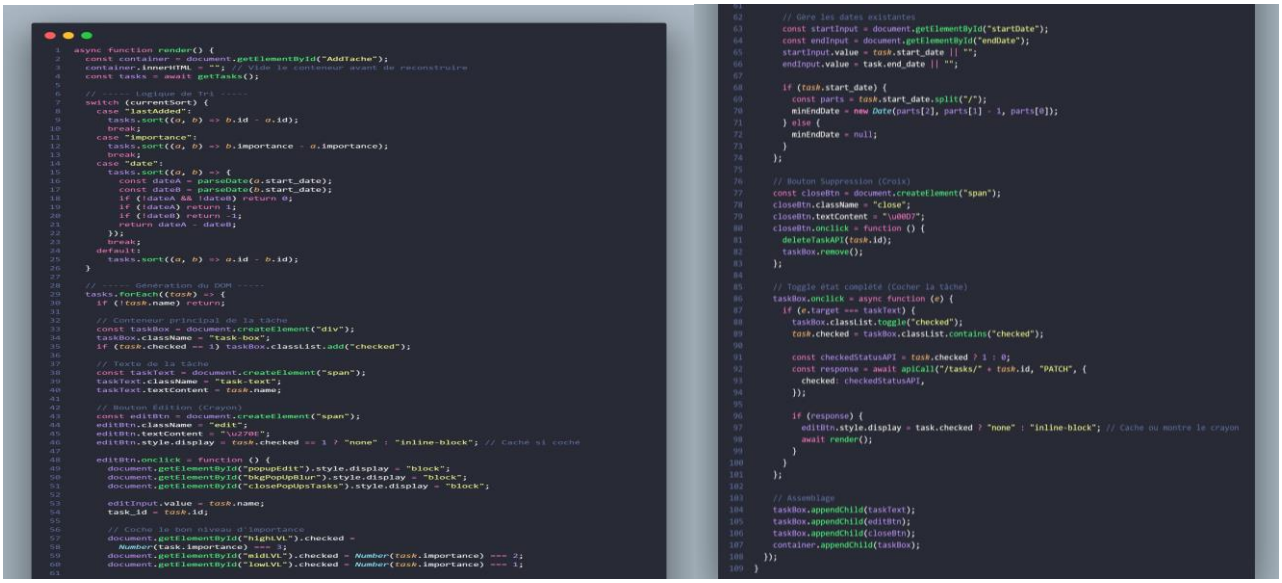
- **Logique :** Récupère le texte tapé et l'ID de l'utilisateur connecté. Génère la date du jour au format "DD/MM/YYYY", puis envoie le tout à la base de données via une requête API POST.
- En cas de succès, le champ de saisie est vidé et la liste des tâches se met à jour en appelant render().

Récupérer les tâches

```js
async function getTasks() {
  const userId = localStorage.getItem("currentUserId");
  if (!userId) return []; // Retourne vide si non connecté
  return await apiCall("/users/" + userId + "/tasks");
}
```

- Lit le localStorage pour trouver l'ID de l'utilisateur actuellement connecté.
- Si l'utilisateur n'est pas connecté, elle retourne simplement un tableau vide. S'il est connecté, elle interroge la base de données (requête GET) pour récupérer sa liste de tâches personnelle.

Tri des tâches & affichage



- Vide le conteneur HTML et trie ensuite le tableau des tâches (par date, importance, ou ID) grâce à un switch.
- Construit dynamiquement les éléments visuels de chaque tâche (texte, bouton d'édition, croix). Gère aussi le clic pour marquer une tâche comme terminée (mise à jour de la classe CSS et requête API PATCH).

Modifier une tâche



```
async function changeTaskPropretys(
  taskId,
  newTask,
  importance,
  startDates,
  endDates,
) {
  const newTaskProprety = {
    name: newTask,
    importance: importance,
    startDate: startDates,
    endDate: endDates,
  };

  const response = await apiCall("/tasks/" + taskId, "PUT", newTaskProprety);

  if (response) {
    showAlert("Task modified successfully!");
    closeModifTask();
    await render();
  }
}
```

```
const btnModif = document.getElementById("modifBtn");
btnModif.onclick = (e) => {
  e.preventDefault();
  const newTask = document.getElementById("textTaskModif").value;
  const importance = impLevels();
  const startDates = document.getElementById("startDate").value;
  const endDates = document.getElementById("endDate").value;

  changeTaskPropretys(task_id, newTask, importance, startDates, endDates);
};
```

- Clic sur le bouton "Edit" depuis le formulaire de modification.
- L'événement récupère les nouvelles données saisies par l'utilisateur (nom, importance, dates) et empêche le rechargement de la page (e.preventDefault()). Il appelle ensuite changeTaskPropretys() qui envoie une requête API PUT pour écraser les anciennes données.
- Affiche une alerte de succès, ferme le popup et rafraîchi la page.

Supprimer une tâche



```
async function deleteTaskAPI(task_id) {
  await apiCall("/tasks/" + task_id, "DELETE");
  render();
}
```

- Prend l'identifiant unique de la tâche sélectionnée et l'envoie à la base de données via une requête API DELETE.
- La fonction render() est appelée immédiatement après pour actualiser la liste à l'écran (fait disparaître la tâche).

Création d'un compte



```
btnInscrire.onclick = () => {
  const username = document.getElementById("usernameCreate").value.trim();
  const email = document.getElementById("userCreate").value.trim();
  const password = document.getElementById("pwdCreate").value.trim();

  // Vérification de base
  if (!username || !password) {
    showAlert("A username and password are required!");
    return;
  }

  creatAccountFunction(username, email, password);
};

// Requête API pour créer l'utilisateur
async function creatAccountFunction(username, email, password) {
  const newAccount = {
    username: username,
    email: email,
    password: password,
  };

  const response = await apiCall("/users", "POST", newAccount);

  if (response) {
    creatToLoginPopup(); // Renvoie vers l'écran de login
    showAlert("Account created!");
  } else {
    const errorData = await response;
    showAlert("Error creating account: " + (errorData.error || "Retry..."));
  }
}
```

- Clic sur le bouton "Create Account".
- Vérification des champs obligatoires (nom d'utilisateur et mot de passe) pour garantir qu'ils ne sont pas vides. Si tout est bon, creatAccountFunction() crée un objet avec les identifiants et l'envoie à l'API via une requête POST.
- Si le compte est créé avec succès, l'utilisateur est basculé vers la fenêtre de connexion (creatToLoginPopup()). Sinon, un message d'erreur est affiché.

Login



```
const btnLog = document.getElementById("logAccountBTN");

btnLog.onclick = () => {
  const userMail = document.getElementById("userMailLOG").value;
  const passwordLOG = document.getElementById("pwdLOG").value;
  loginAccountFunction(userMail, passwordLOG);
};

// Requête API pour se connecter
async function loginAccountFunction(userMail, passwordLOG) {
  const loginAPI = {
    username: userMail,
    password: passwordLOG,
  };

  const response = await apiCall("/login", "POST", loginAPI);

  if (response) {
    // Sauvegarde de la session dans le navigateur
    localStorage.setItem("currentUserId", response.user.id);
    localStorage.setItem("currentUsername", response.user.username);

    // Recharge l'état de la page
    window.onload();

    showAlert("You have successfully connected!");

    // Nettoyage et mise à jour de l'UI
    document.getElementById("pwdLOG").value = "";
    document.getElementById("logoutS").style.display = "block";
    document.getElementById("loginBtn").style.display = "none";

    render(); // Rafraîchit les tâches
    croixCreateAcc(); // Ferme les popups
  } else {
    showAlert("Incorrect login details. Please try again.");
    document.getElementById("pwdLOG").value = "";
  }
}
```

- Clic sur le bouton "Login".
- La fonction envoie les identifiants (mail/mot de passe) via une requête API POST. En cas de réponse positive, l'ID et le nom de l'utilisateur sont enregistrés localement dans le navigateur (localStorage) pour maintenir sa session.
- La page s'actualise (window.onload()), le bouton de déconnexion apparaît, le menu de connexion disparaît, et les tâches de l'utilisateur sont chargées (render()).

Log out



```
const logoutBtn = document.getElementById("logoutS");

logoutBtn.onclick = () => {
  // Suppression de la session
  localStorage.removeItem("currentUserId");
  localStorage.removeItem("currentUsername");

  window.onload(); // Actualise l'interface
  showAlert("You have been disconnected");

  render(); // Vide la liste de tâches
  document.getElementById("logoutS").style.display = "none";
};
```

- Clic sur le bouton de déconnexion.
- Nettoie la session en cours en supprimant l'ID et le nom d'utilisateur du navigateur (localStorage.removeItem()).
- L'interface se réinitialise (window.onload()), une alerte confirme la déconnexion, la liste des tâches à l'écran est vidée, et le bouton de déconnexion est masqué.

Conclusion

Ce projet m'a permis d'approfondir mes connaissances en JavaScript en manipulant la création, la modification et la suppression d'éléments en temps réel, en gérant des requêtes, et en intégrant des fonctionnalités comme le mode sombre ou les animations de texte.

J'ai aussi appris comment modéliser une base de données, et comment utiliser le PHP pour traiter les requêtes.